Mi Red Social

Proyecto Final — Desarrollo Web (UMG). Aplicación web social para almacenar y navegar en fotografías (estilo Instagram): login tradicional + Google + reCAPTCHA, feed de miniaturas, subida de fotos con transformaciones y reconocimiento facial automáticos, y almacenamiento en la nube (Amazon S3).

Integrantes del grupo — Universidad Mariano Gálvez de Guatemala, curso Desarrollo Web:

Integrante	Carné
Karla Gabriela Mendoza Bravo	5090-23-15243
Katherine Melissa Palacios Salguero	5090-23-682
Daniel Eduardo Samayoa	5090-23-9260
Andrea Paola Chacón Segura	5090-23-1225
Jeffrie Otoniel Argueta López	5090-23-8396

Stack técnico

Capa	Tecnología
Frontend	ReactJS 18 (Vite + React Router) + CSS3
Backend	Node.js + Express
Base de datos	PostgreSQL (relacional) + Sequelize ORM
Autenticación	JWT propio + bcrypt (hash + sal) + Google OAuth2 + reCAPTCHA v2
Procesamiento de imagenes	Sharp (transformaciones) + face-api.js (reconocimiento facial)
Almacenamiento	Disco local (desarrollo) / Amazon S3 (producción)
Despliegue	AWS EC2 (t3.micro, Ohio us-east-2) + PM2 + nginx + DNS DuckDNS + HTTPS (Let's Encrypt)

Documentación

- `docs/ARQUITECTURA.md` — arquitectura general y diagramas.
- `docs/ESQUEMA_BD.md` — esquema de la base de datos relacional.
- `docs/DESPLIEGUE.md` — instrucciones paso a paso para desplegar en AWS.
- `docs/ENTREGA_CATEDRATICO.md` — datos de acceso y pasos para la revisión.
- `docs/README.md` — índice que relaciona cada entregable y manual del enunciado con su archivo.
- `docs/MANUAL_TECNICO.md` — manual técnico: frontend, API REST, variables de entorno y operación.

- `docs/DIAGRAMA_COMPONENTES.md` — diagrama distribuido de componentes.
- `docs/DIAGRAMAS_CLOUD.md` — diagramas de cloud computing (AWS, IAM, flujo de solicitud).
- `docs/PLAN_CAPACITACION_IT.md` — plan de capacitaciones para el equipo de IT.
- `docs/MANUAL_USUARIO.md` — cómo usar la aplicación.
- `docs/SEGURIDAD_BD.md` — manual de seguridad de la base de datos.
- `docs/MANTENIMIENTO_BD.md` — plan de mantenimiento de la base de datos.
- `docs/PLAN_RECUPERACION_DESASTRES.md` — plan de recuperación contra desastres (Disaster Recovery Plan).
- `docs/AUDITORIA_TRANSACCIONES.md` — sistema de auditoría de transacciones.
- `docs/SEGMENTACION_RED.md` — segmentación de red.

Cómo correr el proyecto en local

1. Requisitos

- Node.js 20+
- PostgreSQL (local, o usa `docker compose up -d` que levanta solo la base de datos — ver `docker-compose.yml`)

2. Instalación

```
npm install
cp .env.example .env
# edita .env: como minimo DB_PASSWORD, JWT_SECRET, GOOGLE_CLIENT_ID, RECAPTCHA_*
npm run db:init    # crea las tablas en PostgreSQL
npm run dev        # levanta el servidor con nodemon en http://localhost:4000
```

3. Abrir el frontend (React + Vite)

El frontend está en la carpeta `frontend/` y está hecho con **ReactJS** (Vite + React Router). Integra Google Login y reCAPTCHA v2 dentro de componentes React.

```
cd frontend
npm install
npm run dev        # http://localhost:5173
```

Usa el backend en `http://localhost:4000/api` por defecto; para cambiarlo copia `frontend/.env.example` como `frontend/.env`. Para producción: `npm run build` (genera `frontend/dist`, que nginx puede servir como sitio estático).

3b. Frontend anterior (HTML + JavaScript)

3c. Abrir el frontend

Abre `index.html` (o `login.html` para empezar) con un servidor estático cualquiera (por ejemplo la extensión "Live Server" de VS Code, o `npx serve .`) — el frontend llama al backend en

`http://localhost:4000/api` (configurable en `config.js`).

Estructura del proyecto

```
Controlador/     Logica de negocio (usuarios, publicaciones, follows, imagenes)
rutas/           Definicion de endpoints REST
modelo/          Modelos Sequelize (PostgreSQL) + asociaciones
servicios/       JWT, almacenamiento (local/S3), pipeline de procesamiento de imagenes
middlewares/     Autenticacion JWT
database/        Conexion Sequelize + esquema SQL de referencia
models/          Pesos del modelo de face-api.js (NO son modelos de datos)
deploy/          Scripts y plantillas para el despliegue en AWS (EC2, S3, IAM)
docs/            Documentacion de arquitectura, BD, despliegue y usuario
imagenes/        Fotos subidas en modo de almacenamiento local
*.html, app.js, css/   Frontend estatico
```

Funcionalidades implementadas

- [x] Bienvenida y autenticación (login/registro) con Google Login y reCAPTCHA
- [x] Feed de miniaturas para usuarios autenticados
- [x] Algoritmo de miniaturas (Sharp) al subir cada foto
- [x] Subida de fotografías
- [x] Reconocimiento facial automático al subir (face-api.js) + análisis manual bajo demanda
- [x] 3 transformaciones automáticas por foto (miniatura, blanco y negro, sepia), + 9 filtros manuales extra
- [x] Visualización en resolución completa con sus transformaciones
- [x] Almacenamiento en la nube (Amazon S3) con capa de abstracción local/S3
- [x] Base de datos relacional (PostgreSQL) para cuentas e índice de imágenes (nunca el binario)
- [x] Contraseñas con hash bcrypt + sal única por usuario
- [x] Artefactos de despliegue para AWS EC2 (PM2, nginx, script de bootstrap)
- [x] Plantillas de política IAM (acceso limitado para el catedrático, acceso S3 de la app)
- [ ] Amazon Rekognition (opcional avanzado, no implementado)